CS-330 Final Project Review

Jessica Soler

Design Decisions

My scene recreates a tabletop arrangement with a wine bottle, cracker box, cheese wheel, chocolate package, and apple. These objects gave me a way to practice different shapes while keeping the scene manageable. I used a plane for the table, cylinders and a tapered cylinder for the bottle, a cylinder for the cheese wheel, a prism for the chocolate, and a flattened sphere with a cylinder stem for the apple. Scaling, rotating, and placement helped to recreate the reference image. Removing unnecessary bottle surfaces reduced its drawn triangle count to 966.

I considered how the objects would look together. I placed the taller cracker box and wine bottle toward the back. The smaller objects were placed in the front so they would be visible. The chocolate package sits at an angle to match the arrangement in the photograph. I made the apple slightly wider and flatter because a perfectly round sphere did not look natural. These adjustments helped the scene resemble the photograph while keeping the individual models simple.

The cracker box was split into six planes so each face could have its own image texture. Applying one texture to the entire chocolate prism distorted the words on the texture. I decided to create a navy prism with a textured front plane instead. I used ChatGPT (OpenAI, 2026) to generate fictional labels for the cracker box, wine label, and chocolate instead of copying existing brand's artwork. I resized and oriented the images in Adobe Photoshop.

I placed four point lights around the scene, including one warm light. Ambient lighting brightens surfaces facing away from the lights. Diffuse lighting shows their shape. Specular

lighting adds highlights (de Vries, n.d.-a). Separate materials make the paper less shiny than the glass and metal. Transforming the surface normals with the model matrix's inverse transpose helped the lighting follow rotated and scaled objects.

W and S keys move forward and backward. A and D keys move sideways. Q and E keys move up and down. Mouse movement changes yaw and pitch without moving the camera's position. Movement uses elapsed frame time to keep travel more consistent across frame rates, following the approach described by de Vries (n.d.-b). Scrolling down slows movement and scrolling up increases it. Pressing the P key selects perspective and the O key selects orthographic projection. The user is able to switch perspectives without resetting the camera. Escape closes the scene. The console displays the controls and updates the camera position, direction, and speed four times per second.

Following my instructor's feedback, I gave each object its own class inside SceneManager. Each constructor stores the object's parts, and RenderScene() calls the objects' Render() methods. DrawSceneObject() applies a part's settings and selects its mesh. The bottle's setters and GetModelMatrix() let all seven parts move, rotate, and scale together. SetTransformations() combines that whole object matrix with each part's local settings. LoadSceneTextures() loops through named texture paths and tags while checking for loading failures. SetShaderTexture() keeps an assigned color when a texture is missing. DefineObjectMaterials() and SetupSceneLights() keep surface and lighting changes separate from object construction. I also added camera logging, replaced repeated settings with constants, and shortened the rendering code in response to feedback.

References

de Vries, J. (n.d.-a). *Basic lighting*. LearnOpenGL. https://learnopengl.com/Lighting/Basic-Lighting

de Vries, J. (n.d.-b). *Camera*. LearnOpenGL. https://learnopengl.com/Getting-started/Camera

OpenAI. (2026). ChatGPT [Generative AI tool]. https://chatgpt.com/